Grid Path Planning: Wavefronts, Navigation Functions and Value Iteration

Contents

1. Overview 2
2. Numbering every cell from the door: the wavefront 4
3. Which moves count: four or eight neighbours, moves or metres 6
4. Walking downhill: the navigation function 8
   - 4.1 The numbers as a valley 8
   - 4.2 One step at a time, from any cell 9
5. Value iteration: the same numbers without a queue 12
6. Between the cells: dynamic programming with interpolation 14
   - 6.1 The robot is between the cell centres 14
   - 6.2 Value iteration with interpolation 17
   - 6.3 The feedback plan between the cells 18
7. A front that spreads in every direction: fast marching 19
   - 7.1 Why the grid's distances are too long 19
   - 7.2 Fast marching 20
   - 7.3 The Eikonal equation 24
8. Summary 26
9. Sources 27
10. Key terms 28

Where this fits:

- **Builds on:** Partial derivatives and gradients; Gradient descent; Memoization; Breadth-first search; Dijkstra's algorithm; Backward and bidirectional search.
- **Leads to:** D* replanning; Real-time heuristic search (LRTA*).
- **Compare with:** Dijkstra's algorithm; Harmonic potentials; Potential fields.

1. Overview

Key point: A route from A* serves one start, and a robot that slips off it must search again. Numbering every free cell with its distance to the door, in one pass from the door, gives the robot a move from any cell: walking downhill on those numbers reaches the door by the shortest route, wherever the robot finds itself.

So far the robot can find its shortest route to the door on its map, the room's occupancy grid (G-2340) of 0.5 m cells, 76 of them free. With moves to any of the 8 neighbouring cells, A* search (G-2577) finds the 3.414 m route from the start cell S to the door's cell G after taking 9 cells off its open list.

The remaining problem: the route is a list of cells for one start, and the robot does not stay on it. Its position is never exact: the corridor robot missed by 0.27 m on average from its commands alone, and its beacon readings by 0.51 m, a whole cell. Figure 1 plays one such slip: the robot's first move falls one cell short, and it lands in a cell the route does not contain.

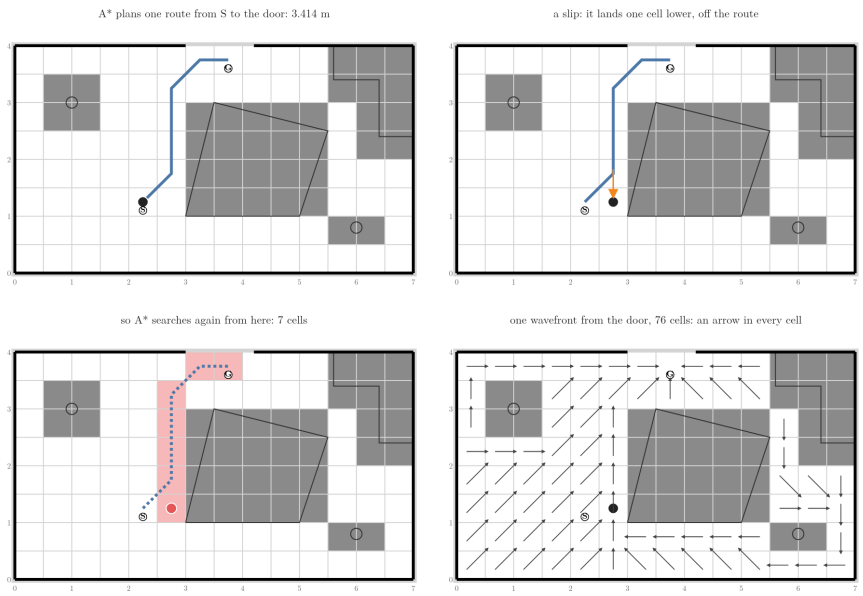


Figure 1: The room's 0.5 m grid (grey: blocked cells, with the true table, cabinet and poles outlined), the start cell S and the door's cell G. A* plans one route from S, 3.414 m (blue). The robot (black dot, drawn at the centre of its cell, as the grid sees it) sets off, but a slip leaves it one cell lower, off the route (orange arrow). The route says nothing about that cell, so A* searches again: 7 cells (light red) and a new route of 3.207 m. The last frame shows what this Note builds: an arrow in every free cell, from one wavefront run from the door

Every slip means another search. Searching from each of the 76 free cells in turn, A* takes 902 cells off its open lists in all (notebook).

One pass from the door, which numbers every free cell with its distance to the door, takes 76, and afterwards a robot in any cell knows its next move. Every number below is computed in the notebook `RO-048-grid-planning-wavefronts-and-value-iteration.ipynb`. This Note builds that pass and the ways to use and compute it:

- numbering every cell from the door, ring by ring: the wavefront (Section 2);
- which neighbours a cell has and what a move costs, which decide what the numbers mean (Section 3);
- walking downhill on the numbers from any cell: the navigation function (Section 4);
- computing the same numbers without a queue, by repeated sweeps: value iteration (Section 5);
- numbers between the cell centres, for a robot that stands anywhere and drives at any angle: dynamic programming with interpolation (Section 6);
- distances that are right in every direction, from a front spreading over the floor: the fast marching method and the Eikonal equation (Section 7).

2. Numbering every cell from the door: the wavefront

> **Key point:** Give the door's cell 0, its free neighbours 1, their unnumbered neighbours 2, and so on. One breadth-first pass from the door numbers all 76 free cells with their fewest moves to the door; S gets 8.

The idea. The opening problem needs, for every cell the robot might land in, its distance to the door. Drop a stone into a pond at the door: the ripple reaches the cells next to the door first, then the cells next to those, ring after ring. If each cell writes down the ring that reached it, every cell knows how many moves it is from the door.

The standard term. Numbering the cells ring by ring outwards from the goal is **wavefront propagation** (G-2700), and the program that does it is a wavefront planner (Choset slides, "The Wave-front Planner"; LaValle §8.2.3). Each ring is a wavefront: the cells first reached at the same number of moves.

The mechanism on our grid. We start with moves left, down, up and right only, the 4-connected grid (G-2544), whose cells are joined to the at most four cells that share a side with them. The steps (Choset slides; LaValle §8.2.3):

1. Give G the number 0 and put it in a list.
2. Take the oldest cell off the list.
3. Give each free neighbour without a number the taken cell's number plus 1, and add it to the end of the list.
4. Go back to step 2 until the list is empty.

The list is a queue (G-2551), so these steps are breadth-first search (G-2550) run as a backward search (G-2583), from the door, and run until every cell is reached instead of stopping at S.

Figure 2 plays the wavefront, one ring per frame. Watch the numbers spread from G along the top wall and down both sides of the table: in frame "ring 8", the ring reaches S.

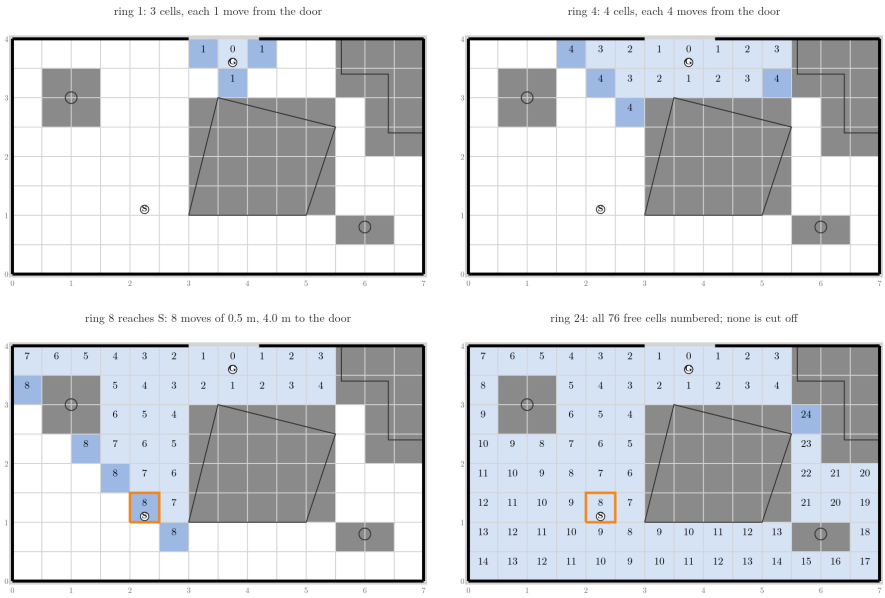


Figure 2: The wavefront on the 4-connected room grid, one ring per frame: breadth-first search from the door's cell G writes in every free cell its fewest moves to G. The ring just numbered is mid blue, earlier rings pale blue. Ring 1 has 3 cells, ring 4 has 4; ring 8 reaches S, 8 moves of 0.5 m, 4.0 m from the door. The last frame, ring 24, has all 76 free cells numbered; none is cut off

S gets the number 8, the same 8 moves that breadth-first search from S found to the door, because a move costs the same in both directions. The pass ends after ring 24, with every free cell numbered. A cell that never gets a number cannot reach the door at all (Choset slides: "0's should only remain if unreachable regions exist"); our room has none.

The formal version. With:

- W_k is the wavefront number k, the set of cells first reached after k moves, such as $W_0 = \{G\}$ (curly brackets list the members of a set: here only G) and W_8, the 5 cells of ring 8, S among them;
- $\phi(x)$ is the number written in cell x, such as 8 for S.

Each wavefront is made of the free neighbours of the last one that have no number yet, and a cell's number is the index of its wavefront (LaValle §8.2.3):

$$\phi(x) = k \quad \text{for every } x \text{ in } W_k$$

Each cell enters the list once and leaves it once, so the work grows in proportion to the number of cells, written $O(n)$ in Big-O notation (G-2556) for n reachable cells (LaValle §8.2.3), 76 here. The numbers count moves along 4 directions, but our robot also moves diagonally, and a diagonal is longer than a straight move. What should a cell count then?

3. Which moves count: four or eight neighbours, moves or metres

> **Key point:** The numbers mean what the moves and their costs say: on the 4-connected grid S is 8 moves (4.0 m) from the door, and on the 8-connected grid 6 moves, but walking down those numbers can take 3.828 m. Counting metres instead, with Dijkstra's algorithm run from the door, gives S its true 3.414 m.

The wavefront of Section 2 numbers each cell with its moves to the door, but the robot drives in metres. Two choices fix what a number means:

- **which cells are neighbours**: the 4-connected grid, or the 8-connected grid (G-2564), which joins each cell to all 8 cells around it (MR §10.4) and, on our grid, never lets a diagonal move cut the corner of a blocked cell;
- **what a move costs**: one move each, or its length, 0.5 m straight and 0.707 m diagonal.

Figure 3 numbers the room three ways, and draws the route from S that walks down each set of numbers. Compare the three numbers in S's cell (orange outline) and the length of each route.

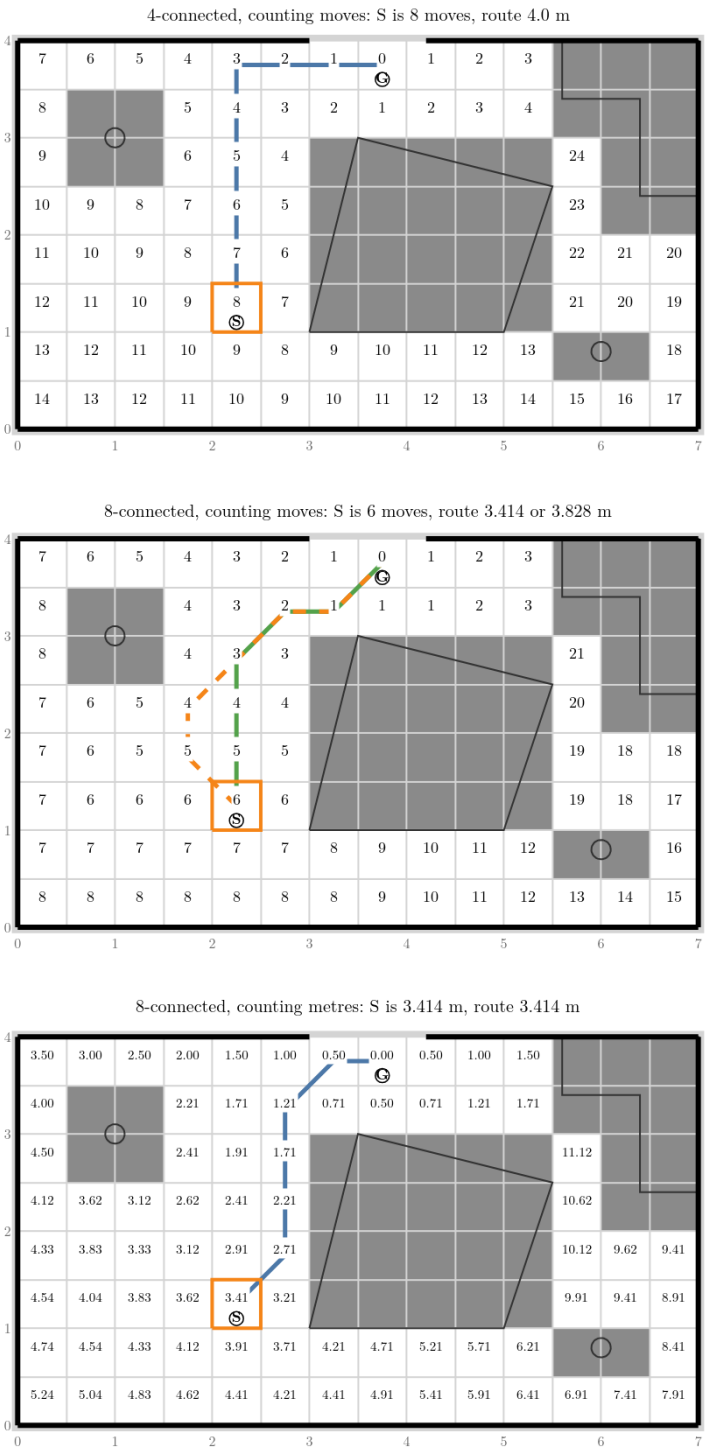


Figure 3: The door's numbers under three choices, on the room grid; each cell holds its number, and the line from S walks to lower numbers until it reaches the door's cell G. Top: 4-connected, counting moves: S is 8 moves from the door, and the route is 4.0 m. Middle: 8-connected, counting moves (a diagonal counts as one move, like a straight one): S is 6 moves away, but a route down the numbers can be 3.414 m (green) or 3.828 m (orange, dashed). Bottom: 8-connected, counting metres, numbers rounded to 0.01 m: S is 3.414 m away, and the route down the numbers is 3.414 m

Counting moves on the 8-connected grid fails. With diagonal moves allowed, S is 6 moves from the door. But two routes of 6 moves can differ in length, the blind spot of breadth-first search: of the 24 walks from S that step to a neighbour one move nearer the door each time, 16 are 3.828 m long and 8 are 3.414 m (notebook). The numbers cannot tell them apart.

Counting metres. To count metres, the pass must settle cells in order of their distance to the door, not of their moves, and that is Dijkstra's algorithm (G-2565) run from the door until its priority queue is empty. The wavefront of Section 2 is the special case in which every move costs the same: then all cells of one ring are equally far and can be settled together, without a priority queue (LaValle §8.2.3). In the bottom panel of Figure 3, S's number is 3.414 m, and the walk down the numbers is the 3.414 m route of A*.

The standard term. A cell's number in metres, the length of the shortest route from that cell to the goal, is its **cost-to-go** (G-2701) (LaValle §2.3.1, §8.2.2). From here on we write it $V(x)$:

- x is a free cell;
- $V(x)$ is its cost-to-go, in m, such as 3.414 for S and 0 for G.

The cost-to-go of a cell is the true distance left that A*'s heuristic (G-2570) only guesses: for S, the octile guess is 3.121 m and the cost-to-go 3.414 m. Every free cell now has its distance to the door; the robot still needs a rule that turns the numbers into moves.

4. Walking downhill: the navigation function

> **Key point:** From any cell, step to the neighbour with the smallest move plus cost-to-go V, the cell's shortest distance to the door; each step lowers V by exactly the move's length, so the walk is as long as the start cell's number: 3.414 m from S, 3.207 m from the cell the slip left the robot in. A number map with this property is a navigation function.

4.1 The numbers as a valley

The opening problem asks for a move from every cell. Picture each cell's cost-to-go as a height: the door's cell at height 0, S at 3.414, the corner by the cabinet at 11.121. The room becomes a valley whose lowest point is the door, and a ball dropped anywhere rolls to the door.

Figure 4 builds that picture and then turns it into the flat map the later figures use. Watch the heights in the first frame: the cells along the top wall, by the door, are lowest; the cells on the far side of the table, which must go round it, are highest. Then the camera tilts until it looks straight down, and only the colours are left: pale yellow is low, blue is high.

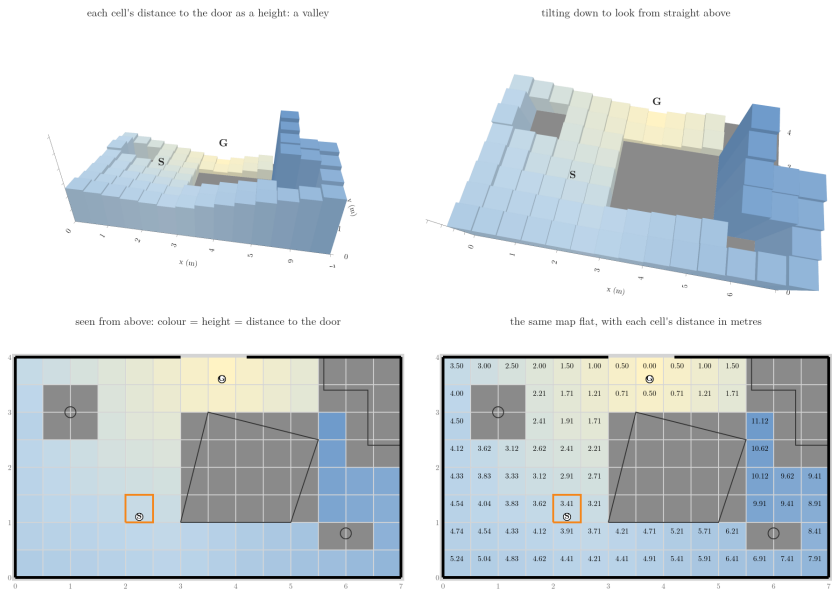


Figure 4: The cost-to-go of every free cell of the room grid as a height: each cell is a column as tall as its distance to the door (0 at G, 3.414 at S, 11.121 at the corner by the cabinet); blocked cells are flat and grey; heights are drawn at about a quarter of the floor's scale. The camera tilts from a side view to straight above. The last frames show the same map flat, each cell coloured by its height (pale yellow low, blue high), and then with its cost-to-go in metres written in it, rounded to 0.01 m

4.2 One step at a time, from any cell

The rule. A robot in a cell looks at its neighbours and steps to the one for which the move plus that neighbour's number is smallest (LaValle §8.2.2, eq. 8.5). From S, with each move's length and the neighbour's cost-to-go (notebook):

Move from S	Move (m)	Neighbour's number (m)	Sum (m)
straight up	0.5	2.914	3.414
diagonally up and right	0.707	2.707	3.414
straight right	0.5	3.207	3.707
diagonally up and left	0.707	3.121	3.828
straight left	0.5	3.621	4.121
straight down	0.5	3.914	4.414
diagonally down and right	0.707	3.707	4.414
diagonally down and left	0.707	4.121	4.828

The smallest, 3.414, is S's own number, reached by two moves; we break the tie towards the neighbour with the smaller number, diagonally up and right.

Why the walk is the shortest route. S's number is the length of its shortest route, and that route's first move leads to a neighbour whose shortest route is the rest of it. With:

- n is a neighbour of the cell x;
- $c(x, n)$ is the length of the move from x to n, 0.5 or 0.707 m;
- min means the smallest of the numbers that follow, here one for every neighbour n;
- n^* is the neighbour the rule picks, diagonally up and right for S.

Our numbers first, then the same in symbols:

$$V(S) = \min(3.414, 3.414, 3.828, \dots)$$

$$V(x) = \min_n \Big(c(x, n) + V(n) \Big)$$

$$V(S) = 0.707 + 2.707 = 3.414$$

$$V(x) = c(x, n^*) + V(n^*)$$

So each step lowers the number by exactly the move just made. The walk from S passes through cells numbered 3.414, 2.707, 2.207, 1.707, 1.207, 0.5 and 0, and its moves add up to the drop from the first number to the last:

$$\text{walk} = 3.414 - 0 = 3.414 \text{ m}$$

$$\text{walk} = V(\text{start}) - V(G)$$

The same telescoping holds from every cell, because every cell's number came from such a neighbour. When every move costs the same, as on the 4-connected grid of Section 2, the move adds the same amount everywhere, and the rule becomes "step to the neighbour with the lowest number" (Choset slides, "The Wavefront, Now What?"; LaValle §8.2.2). On our 8-connected grid the lowest-number rule happens to give shortest routes from all 75 cells (notebook), but only the rule with the move added is guaranteed to.

Figure 5 draws the rule as an arrow in every cell and lets four robots follow the arrows, one after another: from S, from the cell the slip of Figure 1 left the robot in, and from the two bottom corners. Watch each robot arrive and compare the title's length with its start cell's number in Figure 4.

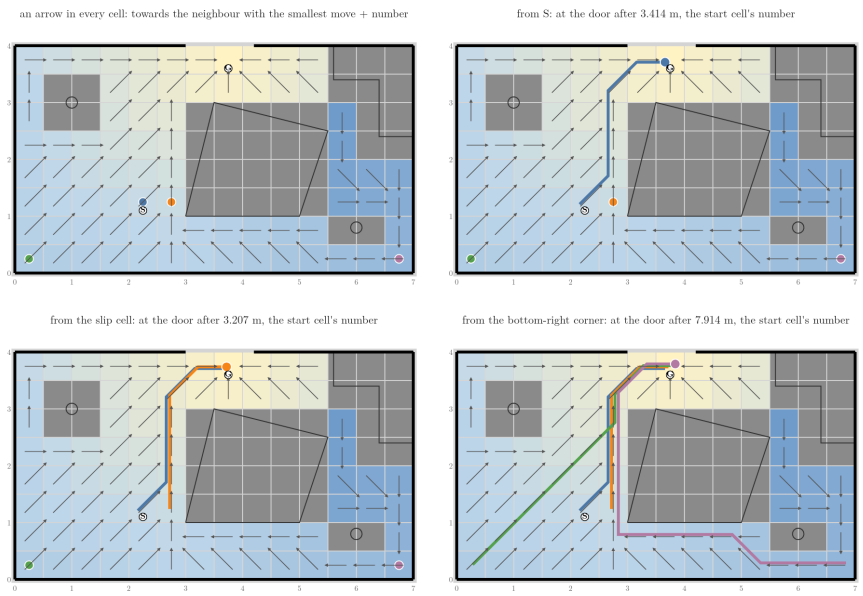


Figure 5: The cost-to-go map of Figure 4 (pale yellow near the door, blue far away) with an arrow in every free cell pointing to the neighbour with the smallest move plus cost-to-go. Four robots follow the arrows one after another, their routes drawn slightly apart where they share cells: from S (blue), 3.414 m; from the cell the slip left the robot in (orange), 3.207 m; from the bottom-left corner (green), 5.243 m; from the bottom-right corner (purple), 7.914 m, round the bottom of the table. Each route's length equals its start cell's number

The slipped robot of Figure 1 no longer searches: its new cell's arrow sends it on a 3.207 m route, the same route A* found by searching 7 cells. The navigation function answers the opening problem: one pass from the door, and every cell, on the route or off it, has its move.

The standard terms. A number map that guides the robot this way is a **navigation function** (G-2702) (LaValle §8.2.2). LaValle's three conditions, on our map:

1. the goal has the value 0: $V(G) = 0$;
2. a cell from which the goal cannot be reached has the value infinity, and only such a cell: our room has none;
3. from every other cell, some move leads to a cell with a strictly lower value: true for all 75 cells (notebook), because every cell's number was made from a neighbour's number plus a move longer than 0.

The third condition says the valley of Figure 4 has one lowest point, the door, and no local minimum (G-1110): no dent other than the goal where a walker could stop. A height map made by hand, such as a potential field, can have such dents. Walking downhill on a smooth valley is gradient descent (G-862); the grid's arrows are its version on cells (LaValle §8.2.2).

A navigation function with its downhill rule is a **feedback plan** (G-2703): a plan that names an action for every state the robot can be in, instead of one sequence of actions for one start (LaValle §8.2.2). The plan is called feedback because the robot's measured cell, fed back from

its sensors, picks the action, so a slip changes the input, not the plan. A route from A* is not a feedback plan: off the route it names no action.

A drawback: shortest routes hug the obstacles. A shortest route bends round every corner as tightly as the grid allows, so a robot that follows it passes close to the furniture (Freiburg path planning, slide 42). Our grid already blocks every cell that furniture touches, which keeps the robot's centre a quarter of a cell from it along the grid lines. Two common remedies take more room, both by treating the obstacles as bigger than they are (Freiburg path planning, slide 43): drop the free cells next to obstacles before numbering, or give those cells a higher move cost, so the numbers prefer open floor.

The numbers so far came from Dijkstra's algorithm, which settles cells strictly in order of distance using a priority queue. The next section computes the same numbers with no order at all, which is the form that carries over to robots between cells.

5. Value iteration: the same numbers without a queue

> **Key point:** Start with 0 at the door and "unknown" everywhere else, then sweep over all cells, each taking the smallest move plus neighbour's value, until a whole sweep changes nothing. On the room grid 8 sweeps give exactly Dijkstra's numbers; no priority queue and no order are needed.

Why we need it. The numbers that sent the slipped robot to the door came from Dijkstra's algorithm, which computes the cost-to-go, each cell's shortest distance to the door, correctly because it settles cells in the right order, the nearest first. Keeping that order costs a priority queue, and some problems have no simple order at all, such as values stored between cells (Section 6) or moves whose result is uncertain. LaValle (§2.3.3) puts it this way: value iteration "extends easily to a much broader class of problems", including continuous state spaces and planning with uncertain outcomes, while a Dijkstra-like order "may be inefficient or impossible in practice to maintain".

The idea. Section 4.2 showed that each cell's number equals the smallest move plus neighbour's number. Turn that equation into a rule for fixing numbers: start from wrong numbers and let every cell repeatedly copy the best offer from its neighbours. Wrong numbers improve with every sweep, and once no cell can find a better offer, the numbers are the cost-to-go.

The standard term. Repeating, over all states, the update "value = smallest move cost plus the next state's value" until the values stop changing is **value iteration** (G-2704) (LaValle §2.3.1, §2.3.2; Freiburg path planning, slide 40). The Reinforcement Learning (RL) Notes use the same update with rewards and with probabilities for where a move ends. With $V(x)$, the cost-to-go of cell x in m, and the move lengths $c(x, n)$ of Section 4.2:

$$V(x) \leftarrow \min_{n} \Big(c(x, n) + V(n) \Big)$$

The arrow means: replace the value on the left by the right side. Before the first sweep

$V(G) = 0$ and every other value is infinity, written ∞: not yet known.

The mechanism on our grid. One sweep visits the free cells row by row, from the door's wall down, left to right in each row, and updates each in place, so a cell already uses the values its neighbours received earlier in the same sweep (Sutton and Barto §4.1, §4.4). Figure 6 shows the values after each sweep. Watch two things:

- frame "sweep 1": 49 cells get a value, S already its final 3.414, because each row passes its values to the row below within the same sweep;
- frame "sweep 3": four values drop (orange), such as the cell x 2.0 to 2.5 m under the top wall, from 1.914 to 1.500.

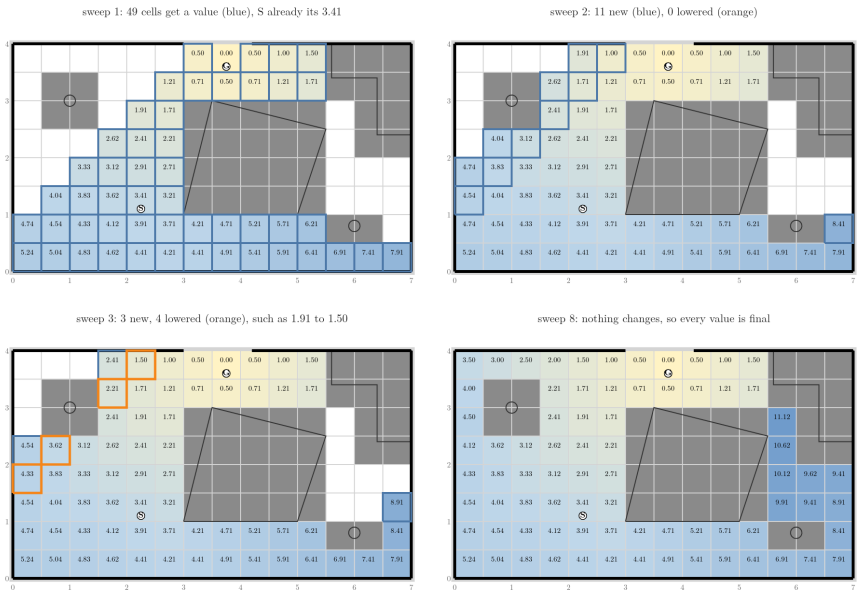


Figure 6: Value iteration on the room grid, one frame per sweep; each cell shows its value in m (rounded to 0.01) and is coloured as in Figure 4; cells with no value yet are white. A sweep goes row by row from the top wall down, left to right. Blue outline: the cell got its first value in this sweep; orange outline: its value was lowered. Sweep 1 gives 49 cells a value, S 3.41; sweep 2 gives 11 more; sweep 3 lowers 4 values, such as 1.91 to 1.50 under the top wall; sweeps 3 to 7 lower 14 values in all. Sweep 8 changes nothing: every value equals the cost-to-go from Dijkstra's algorithm

Why did the cell under the top wall first get 1.914? In sweep 2 it was visited before its right-hand neighbour had a value, so its best offer came diagonally, through a cell with 1.207:

$$0.707 + 1.207 = 1.914$$

In sweep 3 its right-hand neighbour already had 1.0, a shorter way in:

$$0.5 + 1.0 = 1.5$$

Sweep 8 changes nothing, and the 76 values equal Dijkstra's (notebook).

Why the sweeps end at the right numbers. Two facts (LaValle §2.3.2):

1. A value is never too small. Every finite value is the length of some real route to the door, built move by move from G's 0, and no route is shorter than the shortest.
2. After sweep k, every cell whose shortest route has at most k moves holds its exact value. The second cell of such a route has at most $k-1$ moves left, so it was exact after sweep $k-1$, and sweep k offers its value plus the first move.

The longest of the room's shortest routes has 21 moves (notebook), so 21 sweeps always suffice and a 22nd confirms that nothing changes. Sweeps that each use only the values of the last sweep need exactly 22.

With updates in place, a value can travel many cells in one sweep, and here 8 sweeps suffice. According to Sutton and Barto (§4.1), the in-place version "usually converges faster", and the order of the sweep matters.

The price. Value iteration does more work than Dijkstra's algorithm: 8 sweeps of 75 cells are 600 updates, against 76 cells settled once each (notebook). Dijkstra's algorithm is value iteration that only touches the cells whose values can still change, in the order that makes each touch final (LaValle §2.3.3).

Value iteration pays for doing without that order. The next section needs exactly that freedom, because its values sit between the cells.

6. Between the cells: dynamic programming with interpolation

> **Key point:** The robot stands at (2, 1), not at a cell centre, and can drive at any angle. Storing values at the cell centres, blending the four around any point, and running value iteration with moves in 16 directions gives a cost-to-go between the cells; following it, the robot reaches the door's cell after 3.564 m instead of 3.768 m by the grid's cells.

6.1 The robot is between the cell centres

The problem, in numbers. The navigation function of Section 4, the door's numbers the robot walks down, has a value at each cell centre and moves in 8 directions. But the robot stands at (2 m, 1 m), the corner of S, and can turn to any heading. To use the grid it must first drive 0.354 m to S's centre and then follow 45° steps:

$$0.354 + 3.414 = 3.768 \text{ m}$$

To plan from where the robot really is, we need values at points between the centres.

The idea. A point between four cell centres takes a blend of their four values, each counted more the closer the point is to it. Draw the four centres as the corners of a square and the point inside: each corner's share is the area of the rectangle on the far side of the point from that corner, so the nearest corner, with the largest far rectangle, counts most.

The standard term. The blend is **bilinear interpolation** (G-2706) (LaValle §8.5.2.1, eq. 8.59): linear interpolation (G-1092), placing a value between two neighbours in proportion to the distance, done once along x and once along y.

The mechanism on our numbers. Figure 7 takes the end of one 0.5 m move of the robot, at 67.5° from (2, 1): the point p = (2.191, 1.462). The four cell centres around p, with the cost-to-go of Section 6.2, are (1.75, 1.25) with 3.557, S's centre (2.25, 1.25) with 3.338, (1.75, 1.75) with 3.122 and (2.25, 1.75) with 2.862. Look at the blue rectangle: it is the largest, so S's centre, the corner diagonally opposite it, gets the largest weight.

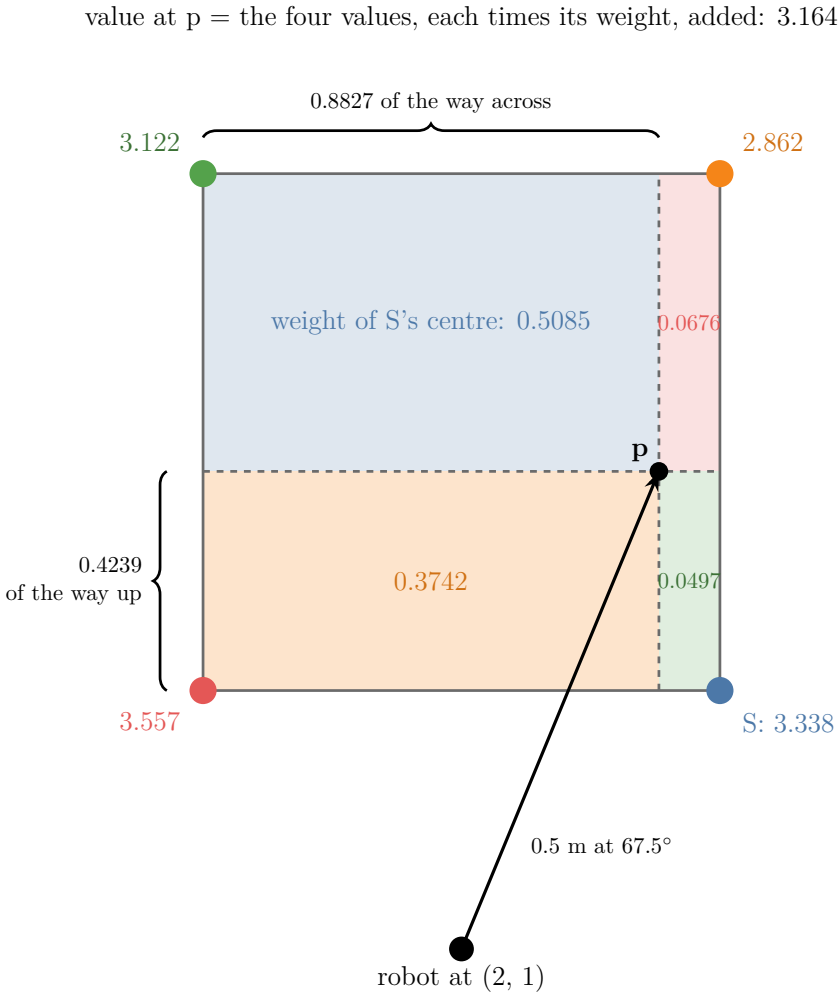


Figure 7: Bilinear interpolation at the end of one move: the robot at (2, 1) moves 0.5 m at 67.5 degrees to p = (2.191, 1.462). The square's corners are the four cell centres around p, with their cost-to-go after dynamic programming with interpolation: 3.557 (bottom left), 3.338 (bottom right, S's centre), 3.122 (top left), 2.862 (top right). p is 0.8827 of the way across and 0.4239 of the way up. Each corner's weight is the area of the rectangle on the far side of p, drawn in that corner's colour: 0.0676, 0.5085, 0.0497 and 0.3742; the value at p is 3.164

How far across and up the square p lies, from the bottom-left centre (1.75, 1.25), in a square of side 0.5 m, with p's coordinates to 5 decimals, (2.19134, 1.46194):

$$\text{across} = (2.19134 - 1.75)/0.5$$

$$= 0.8827$$

$$\text{up} = (1.46194 - 1.25)/0.5$$

$$= 0.4239$$

Each weight is a product of the two shares on the far side (notebook, values to 4 decimals):

$$0.1173 \times 0.5761 = 0.0676$$

$$0.8827 \times 0.5761 = 0.5085$$

$$0.1173 \times 0.4239 = 0.0497$$

$$0.8827 \times 0.4239 = 0.3742$$

The value at p, each corner's value times its weight:

$$0.0676 \times 3.557 = 0.2405$$

$$0.5085 \times 3.338 = 1.6974$$

$$0.0497 \times 3.122 = 0.1552$$

$$0.3742 \times 2.862 = 1.0710$$

$$V(p) = 0.2405 + 1.6974 + 0.1552 + 1.0710$$

$$= 3.164 \text{ m}$$

The formal version. With:

- α is how far across the square the point lies, from 0 at the left centres to 1 at the right ones, 0.8827 for p;
- β is how far up, from 0 at the lower centres to 1 at the upper ones, 0.4239 for p;
- V_{00}, V_{10}, V_{01} and V_{11} are the values at the bottom-left, bottom-right, top-left and top-right centres: 3.557, 3.338, 3.122 and 2.862.

$$V(p) = (1-\alpha)(1-\beta)\,V_{00}$$

$$+\,\alpha(1-\beta)\,V_{10}$$

$$+\,(1-\alpha)\beta\,V_{01} + \alpha\beta\,V_{11}$$

Why it holds: checks at easy points. At the bottom-left centre itself, $\alpha = \beta = 0$, and the first weight is 1 and the others 0, so the blend gives that centre's own value. At the middle of the square, $\alpha = \beta = 0.5$, and every weight is 0.25, so the blend is the plain average. And the four weights always add up to 1, so a blend of four equal values gives that value back:

$$0.0676 + 0.5085 + 0.0497 + 0.3742 = 1$$

$$(1-\alpha+\alpha)(1-\beta+\beta) = 1$$

The robot can now read a value anywhere between four free centres. Where one of the four centres is a blocked cell, LaValle (§8.5.2.2) gives the point the value infinity, so no move ends there.

6.2 Value iteration with interpolation

The idea. Run the value iteration of Section 5 on the cell centres, but let each centre try moves in more than 8 directions. Each move ends between centres, and the bilinear blend of Section 6.1 gives the value at its end. The update is unchanged: the smallest move length plus value at the move's end.

The standard term. Value iteration with values stored at sample points and blended in between is **dynamic programming with interpolation** (G-2705) (LaValle §8.5.2). The method is dynamic programming (G-652), reusing each stored value instead of recomputing routes, on a continuous floor instead of a list of cells.

The mechanism on our grid. Every free centre tries 16 moves of 0.5 m, one every 22.5°; a centre within 0.5 m of the door's centre may also go straight there (notebook). With:

- x is a cell centre, or later any point of the floor;
- u is one of the 16 directions;
- $x + 0.5u$ is the end of a 0.5 m move in direction u;
- $V(x + 0.5u)$ is the bilinear blend there.

$$V(x) \leftarrow \min_u \Big(0.5 + V(x + 0.5u)\Big)$$

The sweeps continue until no value changes by more than a millionth of a metre: 17 sweeps (notebook). S's centre gets 3.338 m, below the grid's 3.414 m, because S's best move now leaves at 67.5°, between the grid's 45° and 90°.

Why not Dijkstra's algorithm here? A move's value now depends on four stored values at once, and on where exactly the move ends, so no single neighbour settles a centre the way Dijkstra's algorithm needs. LaValle (§8.5.2.3) shows a Dijkstra-like version can be built, with more bookkeeping; value iteration needs none.

6.3 The feedback plan between the cells

The values now exist between the cells, so the downhill rule of Section 4.2 works from any point: try the 16 moves, and take the one with the smallest 0.5 m plus the value at its end (LaValle §8.5.2.2, "Using G* as a navigation function"). The plan names a move for every point of the floor, not only for every cell: a **feedback plan** (G-2703) over continuous positions, built by dynamic programming with interpolation.

Figure 8 lets the robot follow it from (2, 1). In the first frame, the 16 grey dots are the ends of the moves it tries; the orange one, at 67.5°, is chosen, with the smallest total of the 16 (notebook):

$$0.5 + 3.164 = 3.664$$

The next best are 45° with 3.711 and 90° with 3.720. Then watch the robot cut between the cells instead of stepping along them.

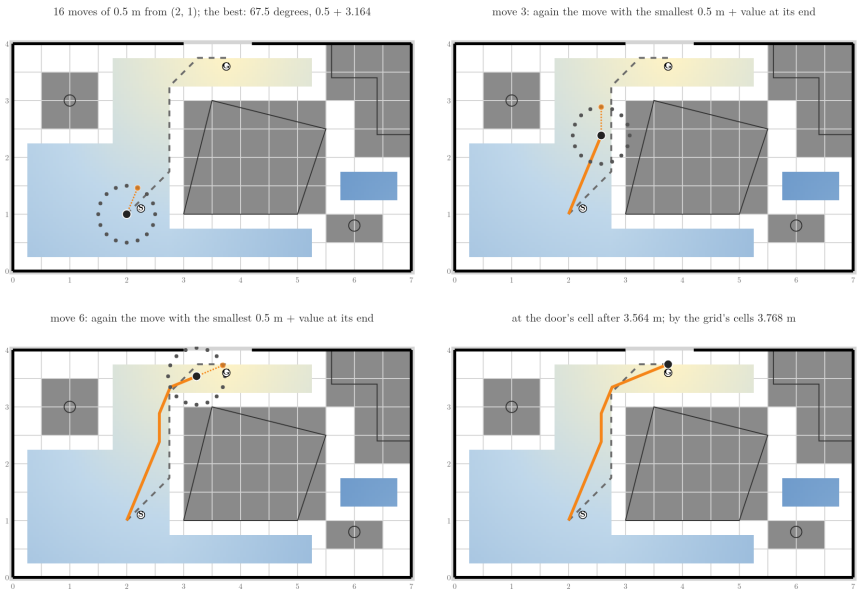


Figure 8: The robot at its true position (2, 1), the corner of S, following the feedback plan from dynamic programming with interpolation, one move per frame. The background is the interpolated cost-to-go (pale yellow near the door, blue far away); white areas are points where one of the four surrounding cell centres is blocked, where no move may end. Grey dots: the ends of the 16 moves of 0.5 m the robot tries; orange: the one with the smallest 0.5 m plus value at its end. The first move goes at 67.5 degrees, 0.5 + 3.164. The robot reaches the door's cell centre after 3.564 m (orange line); by way of the grid's cells, first to S's centre and then along the 8-connected route, it is 3.768 m (grey, dashed)

The route from the robot's true position is 3.564 m, against 3.768 m by the grid's cells (notebook). More directions help further: with 32 moves, one every 11.25°, the route is 3.425 m. The robot can now plan from anywhere on the floor and drive at any angle. The price is 17 sweeps of value iteration, and the values are still built from moves in a few fixed directions. Section 7 shows how far fixed directions put a grid's distances off, and a one-pass method that is not tied to them.

7. A front that spreads in every direction: fast marching

> **Key point:** Dijkstra's distances on a grid follow only the 8 grid directions, so in some directions they come out about 8 percent too long, however small the cells. The fast marching method lets a front spread over the floor at 1 m/s and takes each cell's arrival time from two neighbours at once, so the front can cross the grid at any angle; its error shrinks as the cells shrink.

7.1 Why the grid's distances are too long

The cost-to-go of Section 3, each cell's shortest distance to the door, counts only routes made of the 8 grid moves. An open floor shows the error without obstacles in the way: there, a

point 4 m across and 2 m up from the start is, in a straight line:

$$\sqrt{4^2 + 2^2} = 4.472 \text{ m}$$

The 8-connected grid must reach it by diagonals and straight moves: diagonals that cover 2 m across and 2 m up, 2.828 m long, then 2 m straight:

$$2.828 + 2.0 = 4.828 \text{ m}$$

The grid's route is 8.0 percent too long. Smaller cells do not help: the same mix of diagonal and straight moves is needed at any cell size, so the error stays 8.0 percent (notebook). Sethian puts it this way: with Dijkstra's method on a grid, "no matter how much you refine the mesh, you will still get stairstep patterns: you can never find the diagonal".

In the room, on cells of 0.1 m, the 8-connected distance from the door's cell centre to S's centre is 3.180 m, while the shortest way round the blocked cells, a straight line to the corner of the blocked cells at (3, 3) and on to S's centre, is 2.965 m (notebook).

7.2 Fast marching

The idea. Put a person with a watch at the centre of every cell, and let a front leave the door's cell at 1 m/s in every direction, like a ripple on open water (Sethian). Each person writes down the time the front passes. The recorded time is the cell's cost-to-go in seconds, and because the front moves through the open floor rather than along grid moves, it can arrive from any angle.

A program cannot let a front move, so it builds the times outwards from the door, smallest first, like the oil-depot fire of Dijkstra's algorithm (Sethian: "very closely related to Dijkstra's method"). The one change: a cell's time is worked out from two known neighbours at once, one across and one above or below, so the front can arrive between them on a slant.

The standard term. The method is the **fast marching method** (G-2707) (Sethian 1996). Its loop is Dijkstra's: the cell with the smallest tentative time is accepted and never changed, and its neighbours' tentative times are recomputed (Wikipedia, Fast marching method). Only the update differs.

Figure 9 runs both on the room, cut into cells of 0.1 m, from the door's cell centre. The coloured area is where the front has arrived, coloured by arrival time; the lines are the front every 0.5 s, the cells reached at the same time, like the rings of Figure 2. Watch the shape of the lines in frame "t = 2.50 s": straight sides and corners in the top panel, round arcs in the bottom one.

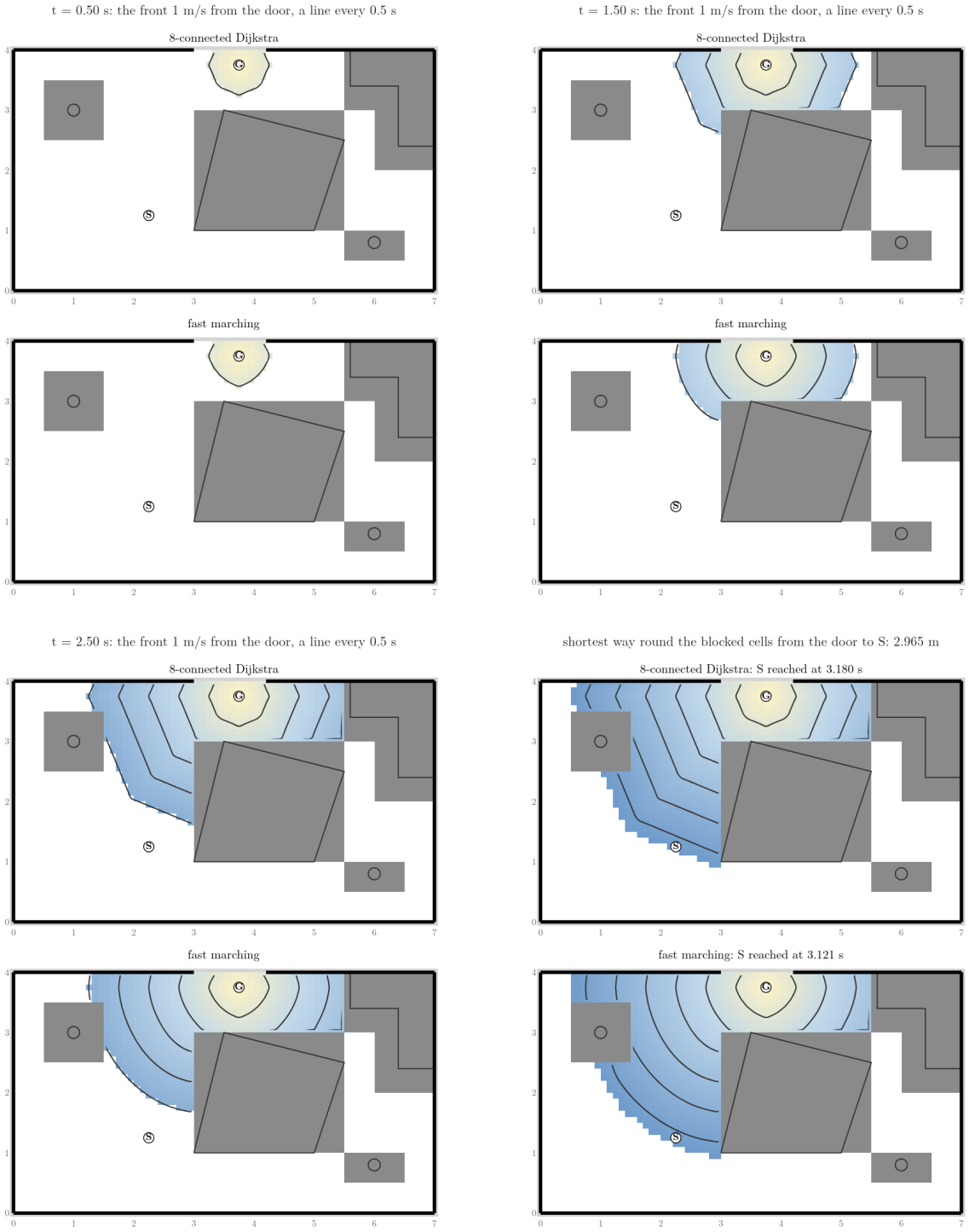


Figure 9: A front leaving the door's cell centre at 1 m/s, on the room's free cells cut into 0.1 m cells; coloured: cells the front has reached, by arrival time (pale yellow early, blue late); dark lines: the front every 0.5 s. One frame per 0.25 s. Top: 8-connected Dijkstra, whose fronts are octagons with straight sides, because its distances follow only the 8 grid directions. Bottom: the fast marching method, whose fronts are round arcs. The last frame gives the arrival times at S's centre: 3.180 s (Dijkstra) and 3.121 s (fast marching); the shortest way round the blocked cells is 2.965 m

The update on our numbers. At S's 0.1 m cell, the neighbour to the right was reached at 3.0807 s and the one above at 3.0288 s (notebook). With:

- a is the smaller arrival time of the cell's left and right neighbours, 3.0807 s;
- b is the smaller arrival time of its lower and upper neighbours, 3.0288 s;
- h is the cell side, 0.1 m;
- F is the front's speed, 1 m/s;
- T is the cell's arrival time, in s.

The update (Wikipedia, Eikonal equation, numerical approximation), for $F = 1$:

$$T = \frac{a + b + \sqrt{2h^2 - (a - b)^2}}{2}$$

At S's cell:

$$a + b = 6.1095$$

$$a - b = 0.0519$$

$$2h^2 - (a - b)^2 = 0.02 - 0.0027$$

$$= 0.0173$$

$$\sqrt{0.0173} = 0.1315$$

$$T = (6.1095 + 0.1315)/2$$

$$= 3.1205 \text{ s}$$

When only one of the two is known, or they differ by at least h, the front came straight along one axis, and the update is the plain move: the smaller of a and b, plus h.

Why the update holds: a check at an easy front. Let a straight front cross the grid at 45°, reaching the left and the lower neighbour together, at 1.0 s. The cell's centre lies a further half diagonal of a 0.1 m cell beyond the line through those two centres, 0.0707 m. Our numbers first, then the same steps in symbols:

$$(T - 1.0)^2 + (T - 1.0)^2 = 0.1^2$$

$$(T - a)^2 + (T - a)^2 = h^2$$

$$2(T - 1.0)^2 = 0.01$$

$$2(T - a)^2 = h^2$$

$$T - 1.0 = 0.0707$$

$$T - a = h/\sqrt{2}$$

$$T = 1.0707 \text{ s}$$

$$T = a + h/\sqrt{2}$$

The update gives exactly the front's true arrival. The same holds for a straight front at any angle, because each of the two slopes is then the same from cell to cell; Dijkstra's distances are exact only along the 8 grid directions. The first line is the equation the update solves, explained in Section 7.3.

The result. At S's centre, fast marching gives 3.121 s, Dijkstra 3.180 s, against 2.965 m by the shortest way round the blocked cells. The difference shows on finer cells (notebook):

Cell side	Fast marching	8-connected Dijkstra
0.1 m	3.121 s	3.180 s
0.05 m	3.061 s	3.151 s
0.025 m	3.021 s	3.136 s

On the open floor of Section 7.1, against the true 4.472 m (notebook):

Cell side	Fast marching	8-connected Dijkstra
0.5 m	4.751 m, 6.2 percent long	4.828 m, 8.0 percent long
0.1 m	4.563 m, 2.0 percent long	4.828 m, 8.0 percent long
0.05 m	4.526 m, 1.2 percent long	4.828 m, 8.0 percent long
0.025 m	4.503 m, 0.7 percent long	4.828 m, 8.0 percent long

Fast marching's error shrinks as the cells shrink, which is the property Sethian's method was built for; Dijkstra's stays at 8.0 percent. On coarse cells fast marching gains little: on the 0.5 m grid of the room it gives S 3.450 s, worse than Dijkstra's 3.414 (notebook).

The update is exact only where the front is nearly straight across a cell, and near the door the front is a circle only a few cells wide. Fast marching is therefore the better choice on fine grids. Like Dijkstra's algorithm, it accepts each cell once and keeps a heap of tentative times, so it takes about $N \log N$ steps for N cells (Sethian).

7.3 The Eikonal equation

What equation does the update solve? Picture the arrival time T, the cost-to-go of Figure 4 measured in seconds, as a height over the floor: the valley of Figure 4, made smooth. The fronts of Figure 9 are its level lines, the points at one height, seen from above, as the rings of Figure 2 were: Figure 9 is a contour plot (G-468) of the arrival-time surface.

Why the steepest slope is 1/F. The left half of Figure 10 shows the front at time T and a short time Δt later. The front moves at speed F, straight across itself, so the two lines are $F\Delta t$ apart. Crossing that gap, the arrival time climbs by Δt:

$$\text{slope across the front} = \frac{\Delta t}{F\,\Delta t}$$

$$= \frac{1}{F}$$

Across the front is the steepest way up, because along the front T does not change at all. The steepest slope of a function and its direction make its gradient ∇T (G-1295): the arrow of its slopes along x and along y, whose length, written with bars, $|\nabla T|$, is the steepest slope (G-1888). So:

$$|\nabla T| = \frac{1}{F}$$

The standard term. The arrival time's steepest slope equal to one over the front's speed everywhere, with $T = 0$ on the goal, is the **Eikonal equation** (G-2708) (Wikipedia, Eikonal equation; Sethian). With $F = 1$ m/s, T in seconds equals the distance to the goal in metres, so the Eikonal equation describes the true shortest distance.

A check on the open floor. From the start of Section 7.1, the arrival time at a point x m across and y m up is its straight-line distance, divided by $F = 1$. At the point (4, 2), $T = 4.472$ s, the square root of 20, and the gradient points straight away from the start, with slope 4 / 4.472 along x and 2 / 4.472 along y:

$$|\nabla T|^2 = \frac{4^2}{20} + \frac{2^2}{20}$$

$$= 0.8 + 0.2 = 1$$

On the grid. The right half of Figure 10 shows the update's version: the slope along x is $(T - a)/h$, from the neighbour across; the slope along y is $(T - b)/h$, from the neighbour above or below. By Pythagoras' rule, their squares add up to the square of the steepest slope:

$$\left(\frac{T-a}{h}\right)^2 + \left(\frac{T-b}{h}\right)^2 = \frac{1}{F^2}$$

Solving this for T gives the update of Section 7.2, and the 45° check there is this equation with $a = b$.

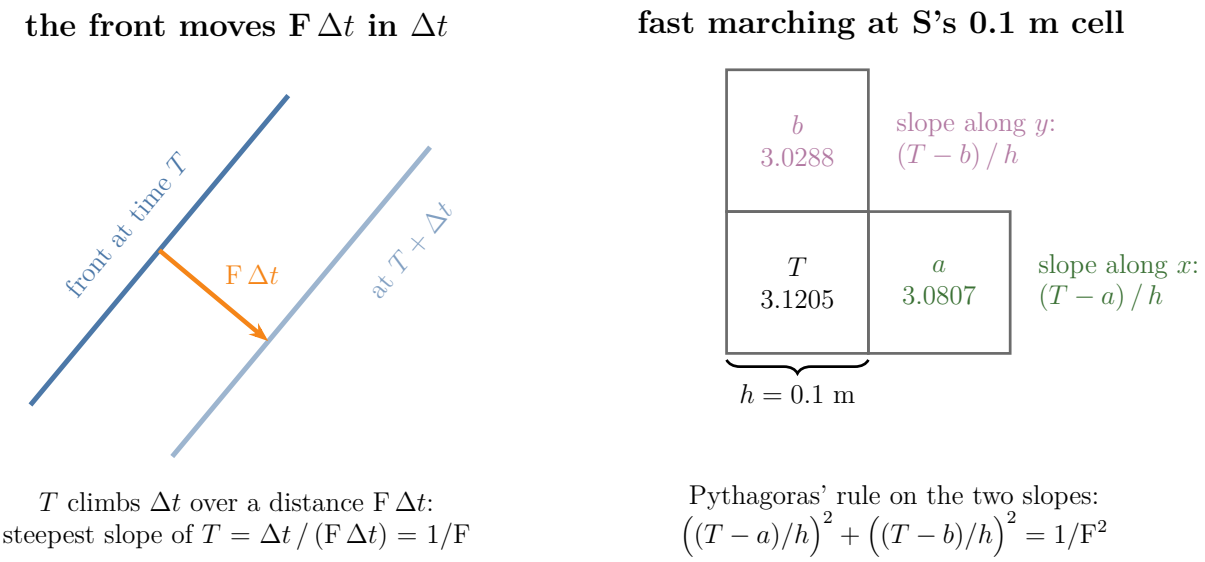


Figure 10: Left: the front at time T (dark blue) and at T + Δt (light blue), a distance F Δt apart (orange arrow), F being the front's speed; T climbs Δt over a distance F Δt, so its steepest slope is Δt / (F Δt) = 1/F. Right: fast marching at S's 0.1 m cell: the neighbour across (right) was reached at a = 3.0807 s, the neighbour above at b = 3.0288 s; the slope of T along x is (T − a)/h and along y is (T − b)/h, with h = 0.1 m; Pythagoras' rule on the two slopes gives the grid equation ((T − a)/h)² + ((T − b)/h)² = 1/F², here T = 3.1205 s

Fast marching on a 4-connected stencil, using only the cells left, right, above and below, thus gives distances in every direction: the grid's connectivity no longer limits the directions the front can take.

8. Summary

Method	What each cell gets	How	S (or the robot)	Why use it
wavefront	its fewest moves to the door	breadth-first search from the door	8 moves (4-connected)	one pass, no queue, when every move costs the same
Dijkstra from the door	its cost-to-go in metres	settle cells nearest first	3.414 m	exact numbers for unequal move costs
value iteration	the same cost-to-go	sweep all cells until nothing changes	3.414 m after 8 sweeps	no queue and no order; carries over to values between cells
dynamic programming with interpolation	a cost-to-go at every point	value iteration, 16 moves, values blended between centres	3.564 m from (2, 1)	plans from where the robot is, at any angle
fast marching	its arrival time	Dijkstra's loop, update from two neighbours	3.121 s on 0.1 m cells	error shrinks with the cells; 8-connected stays 8 percent long

Here the cost-to-go $V(x)$ is the length of the shortest route from cell x to the door, and the arrival time T is the same distance in seconds for a front moving at 1 m/s.

- A route serves one start, so the robot that slipped needed a new A* search (7 cells), and all 76 starts would cost 902 cells; one pass from the door, 76 cells, numbers every cell, and the slipped robot walks 3.207 m downhill without searching, because every cell carries its own distance to the door.
- **Wavefront propagation** (G-2700) numbers the cells ring by ring from the door, so each number is the cell's fewest moves to the door: 8 for S on the 4-connected grid.
- Counting moves on the 8-connected grid gives S 6 moves, but 16 of the 24 walks down those numbers are 3.828 m long, because a diagonal counts like a straight move; counting metres, with Dijkstra's algorithm from the door, gives the **cost-to-go** (G-2701), 3.414 m for S.
- Stepping to the neighbour with the smallest move plus cost-to-go lowers the number by exactly each move, so the walk from any cell is as long as its number; such a number map, 0 at the goal and with a lower neighbour everywhere else, is a **navigation function** (G-2702), and with its rule it is a **feedback plan** (G-2703), a move for every state.
- **Value iteration** (G-2704) gets the same numbers by sweeping "smallest move plus

neighbour's value" over all cells until nothing changes, 8 sweeps here, because a value is never too small and each sweep makes every cell exact whose shortest route has one more move; it does more work than Dijkstra (600 updates against 76) but needs no order.

- **Bilinear interpolation** (G-2706) blends the four centre values around a point, weighted by the far rectangles, so values exist between the cells: 3.164 at the end of the robot's first move.
- **Dynamic programming with interpolation** (G-2705) runs value iteration with 16 move directions and blended values, so the robot plans from (2, 1) at any angle: 3.564 m instead of 3.768 m by the grid's cells.
- The **fast marching method** (G-2707) is Dijkstra's loop with an update from two neighbours, so its front crosses the grid at any angle: 3.121 s at S on 0.1 m cells against Dijkstra's 3.180, with an error that shrinks with the cells while Dijkstra's stays 8 percent.
- The update solves the grid form of the **Eikonal equation** (G-2708), steepest slope of the arrival time T equal to one over the speed F, because the front covers $F\Delta t$ while T climbs Δt.

The robot can now move to the door from any cell or any point of its map, with no search after a slip. The method still assumes that the map is correct. A box left on the floor where the map shows free cells makes some numbers wrong, and the robot sees the box only when it is close; replanning with D* repairs only the numbers the box changes.

9. Sources

Built from

- PDN - PasDeNom, "How do vector field Pathfinding algorithm work?", YouTube, https://www.youtube.com/watch?v=ZJZu3zLMYAc
- Northwestern Robotics, "Modern Robotics, Chapter 10.4: Grid Methods for Motion Planning", YouTube, https://www.youtube.com/watch?v=kCZAgL3jdxk
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.4 "Grid Methods" (4-connected and 8-connected grids). Free preprint: http://modernrobotics.org (MR §10.4)
- Choset, H. (Carnegie Mellon University), *16-735 Robotic Motion Planning*, slides "Potential Functions": "The Wave-front Planner", "The Wavefront, Now What?", https://www.cs.cmu.edu/~motionplanning/lecture/Chap4-Potential-Field_howie.pdf (Choset slides)
- Burgard, W. et al. (Uni Freiburg), *Introduction to Mobile Robotics*, "Path Planning" slides 39 to 43 and 52 (deterministic value iteration on a grid, obstacles and the convolved map), http://ais.informatik.uni-freiburg.de/teaching/ss23/robotics/slides/19-pathplanning.pdf (Freiburg path planning)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §2.3.1 "Optimal fixed-length plans", §2.3.2 "Optimal plans of unspecified lengths", §2.3.3 "Dijkstra revisited", §8.2.2 "Feedback plans as navigation functions", §8.2.3 "Grid-based navigation functions for motion planning" (wavefront propagation), §8.5.2 "Dynamic

programming with interpolation". Free online: https://lavalle.pl/planning/node52.html, https://lavalle.pl/planning/node369.html, https://lavalle.pl/planning/node372.html and https://lavalle.pl/planning/node406.html (LaValle)
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press. §4.1 "Policy evaluation" (in-place updates), §4.4 "Value iteration". Free: http://incompleteideas.net/book/RLbook2020.pdf (Sutton and Barto)
- Sethian, J. A., "Fast Marching Methods: A boundary value formulation", https://math.berkeley.edu/~sethian/2006/Explanations/fast_marching_explain.html (Sethian)
- Wikipedia, "Fast marching method", https://en.wikipedia.org/wiki/Fast_marching_method, and "Eikonal equation", section "Numerical approximation", https://en.wikipedia.org/wiki/Eikonal_eq (Wikipedia, Fast marching method; Wikipedia, Eikonal equation)

Other references

- Sethian, J. A. (1996). "A fast marching level set method for monotonically advancing fronts". *Proceedings of the National Academy of Sciences* 93(4), 1591–1595 (Sethian 1996)
- Likhachev, M. (Carnegie Mellon University), *16-350 Planning Techniques for Robotics*, slides "Search Algorithms: Heuristics, Backward A*, Weighted A* Search": 4-connected and 8-connected grids, https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/heuristicswei

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Wavefront propagation (wavefront planner) (G-2700)	Numbering the free cells of a grid ring by ring outwards from the goal, each ring one move further, so every cell gets its fewest moves to the goal in one pass (8 for the room's start cell); a breadth-first search from the goal, used when every move costs the same.
Cost-to-go (G-2701)	The length of the shortest route from a state to the goal, such as 3.414 m from the room's start cell to the door; stored for every state, it tells the robot how far it still has to go from wherever it is.

Term	Meaning
Navigation function (G-2702)	A number for every state that is 0 at the goal, infinite only where the goal cannot be reached, and lower at some neighbour of every other state, so it has one minimum, the goal; stepping to the neighbour with the smallest move plus number from any state leads to the goal, by the shortest route when the numbers are the cost-to-go.
Feedback plan (G-2703)	A plan that names an action for every state the robot can be in, not one sequence for one start, so the robot's measured state picks the action and a slip only changes the input; a navigation function with its downhill rule is one.
Value iteration (G-2704)	Computing every state's cost-to-go by repeating, over all states, the update "value = smallest move cost plus the next state's value" until no value changes; it needs no priority queue and no order, and it carries over to values between grid cells and to uncertain moves.
Bilinear interpolation (G-2706)	Estimating a value at a point between four grid points as their weighted average, each weight the area of the rectangle on the far side of the point (linear interpolation along x, then along y); it fills in values between cell centres.
Dynamic programming with interpolation (G-2705)	Value iteration with values stored only at sample points, such as cell centres, and blended in between, with moves in many directions; it gives a cost-to-go at every point, so a robot can plan from where it really stands and drive at any angle.
Fast marching method (G-2707)	Dijkstra's loop run on a grid with an update that takes a cell's arrival time from two neighbours at once, one across and one up or down, so a front spreading from the goal can cross the grid at any angle; its distances get closer to the true ones as the cells shrink, unlike 8-connected Dijkstra's.

Term	Meaning
Eikonal equation (G-2708)	The rule that the arrival time T of a front moving at speed F has steepest slope 1/F everywhere, written as the length of its gradient equals 1/F, with T = 0 where the front starts, the goal; its solution with F = 1 is the shortest distance, and fast marching solves its grid form.
Occupancy grid (G-2340)	A grid map that stores in each cell a number between 0 and 1, such as the probability that the cell is occupied, instead of a hard 0 or 1; the standard grid map of robotics, since sensor readings are uncertain.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
4-connected grid (G-2544)	A grid made into a graph by joining each cell to the up to four cells that share a side with it (left, down, up, right); the simplest graph of a grid map, with every move the same length.
Queue (first in, first out) (G-2551)	A list served in arrival order: new items join the back and the next item leaves from the front; as a search's frontier it gives breadth-first search.
Breadth-first search (BFS) (G-2550)	A search whose frontier is a queue, so it expands vertices in rings of equal moves from the start; the first route it finds has the fewest moves, which is the shortest only when every move costs the same.
Backward search (G-2583)	A search that starts at the goal and follows every move backwards until it reaches the start; it is cheaper when the goal's side branches less, and run to the end it gives every state's cost to the goal.

Term	Meaning
Big-O notation (G-2556)	A way to write how fast an algorithm's work grows with the problem's size, keeping only the fastest-growing part and dropping constant factors, such as $O(V + E)$ for breadth-first search and $O(b^d)$ for listing every path; it lets us compare methods without timing them.
8-connected grid (G-2564)	A grid made into a graph by joining each cell to all 8 cells around it: 4 straight moves of one cell side and 4 diagonal moves of 1.414 cell sides (0.707 m for 0.5 m cells), usually without cutting the corner of a blocked cell; it gives shorter, less jagged routes than a 4-connected grid.
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Linear interpolation (G-1092)	Placing a percentile between two neighbouring sorted values, in proportion to its position; the pandas default.
Dynamic programming (G-652)	Solving a problem by storing the answer to each overlapping sub-problem the first time it is computed and looking it up afterwards, so no work is repeated; memoization is one form of it.
Contour plot (G-468)	A map of a surface seen from above, with lines joining points of equal height, so a 3D shape such as a loss bowl can be drawn and read on flat paper.

Term	Meaning
Nabla (∇) (G-1295)	The upside-down triangle ∇, read "nabla" or "grad"; ∇f means the gradient of f, the vector that collects all its partial derivatives.
Steepest ascent (G-1888)	The direction in which f increases fastest: the direction of the gradient; gradient descent steps the opposite way to go downhill fastest.